	Perfect Collision Workshop	
	Subject: Project 1A Classe(s) : 1A Team: Project 1A	Teaching unit: Algorithmic & Programming Academic year: 2020-2021

Objectives:

Be interested in the implementation phase by referring to the C**DIO [IMPLEMENT] initiative:**

- ✓ The concrete implementation of the elements planned during the Design and Design phases
- ✓ Development of necessary functions in C
- ✓ Testing and validation of the work done

1. Principle:

The Collision is a function that takes as parameter 2 objects, or an object and a world made of floors and walls and it will return a boolean value that valle '1' in case of contact between the objects passed as parameters.

Depending on the game story, the collision functions will be different, but will always return "yes" or "no".

In this workshop we will realize a function for the perfect collision.

What's the perfect collision?

How do I set the collision function parameters?

How to implement these collision functions according to our needs?

This is the purpose of this Workshop «Perfect Collision», which will present the steps of preparation of the parameters of the Collisionfunction that will **be coded in C.**

2. Perfect collision detection techniques:

The perfect collision, or we can define it as "pixel-close collisions", is to collect the R, G, and B values of a reference pixel of an entity A in relation to a P plane. If the R, G, and B values of A coincide with those of P, that means there is a collision. This is how the displacement of entity A in plan P can be conditioned.

In order to implement this technique, you should continue with the following steps:

- Create a new background (The Mask) containing the marking of obstacles in a color that does not exist in the original background. In this workshop, we will use pure black color (R:0/ G:0/B:0).
- Loading both backgrounds (original and mask).
- Retrieve RGB values from one or more reference pixels.
- Test the collision at one or more reference pixels depending on the direction.

3. Mask Creation

3.1. Job Aid: GIMP

GIMP is a free open source computer graphics software, offering image and photography processing similar to those offered by Adobe Photoshop.

GIMP stands for GNU (**Image Manipulation Program**). It allows the use of advanced features, such as graphical tool palettes, the use of channels and layers, the management of the main image formats, etc.

3.2. Installation

For Ubuntu machines, the installation of the gimp tool will be done via the **apt-get install**via terminal.

Downloading and installing the GIMP tool on an Ubuntu machine requires an internet connection. To do this type the command:

$ sudo apt-get install gimp

Once installed, you can run gimp with the command:

$ gimp

3.3. Mask background generation:

The use of masks will make it possible to detect collisions on areas of any shape with a very specific color. In our case, we will choose the black color to color the obstacles.

<u>The steps to follow:</u>

- Select the File **tab** , then **Open** to choose the original background

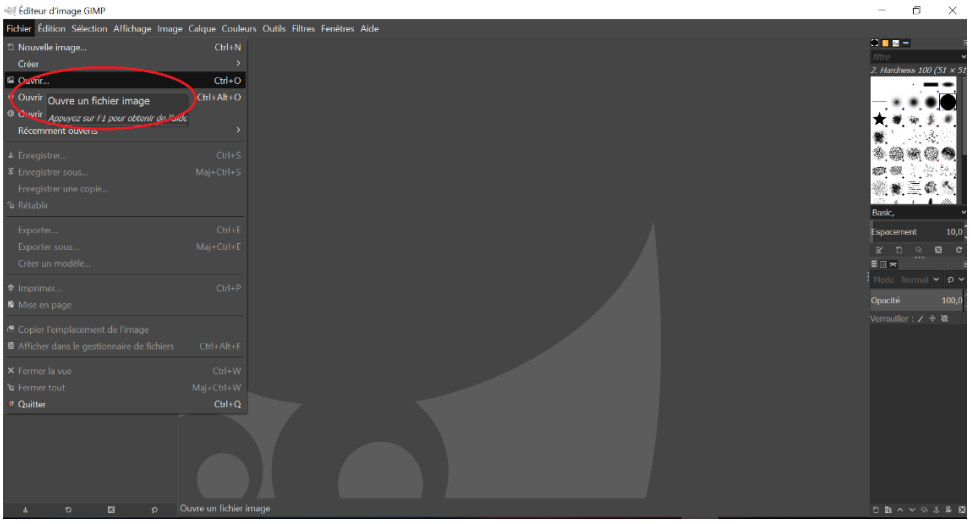


Figure 1: Opening a new image file

- After downloading the image, select the **rectangular selection tool**

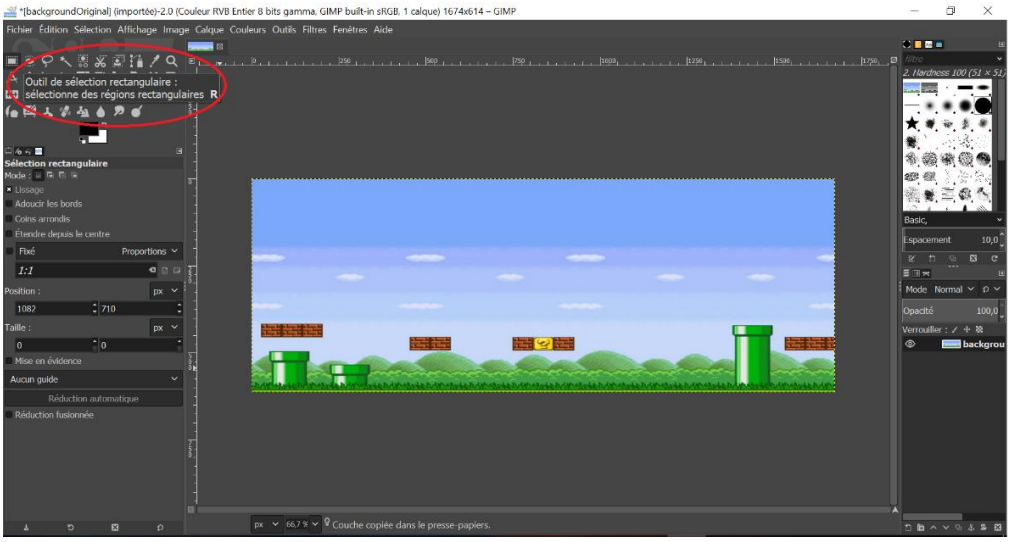


Figure 2: Rectangular selection choice for obstacle selection

- After selecting the obstacles, right click**,select** Colors Color the image **(in our case the selected obstacle)**

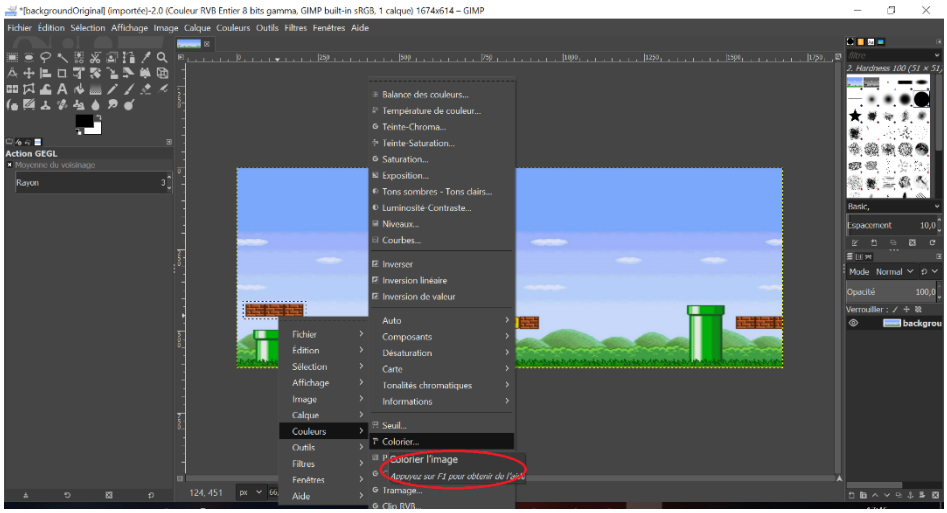


Figure 3: Selecting and Changing Obstacle Colors

- After selecting «Color the **image», choose the black color(R=0,G=0,B=0)**

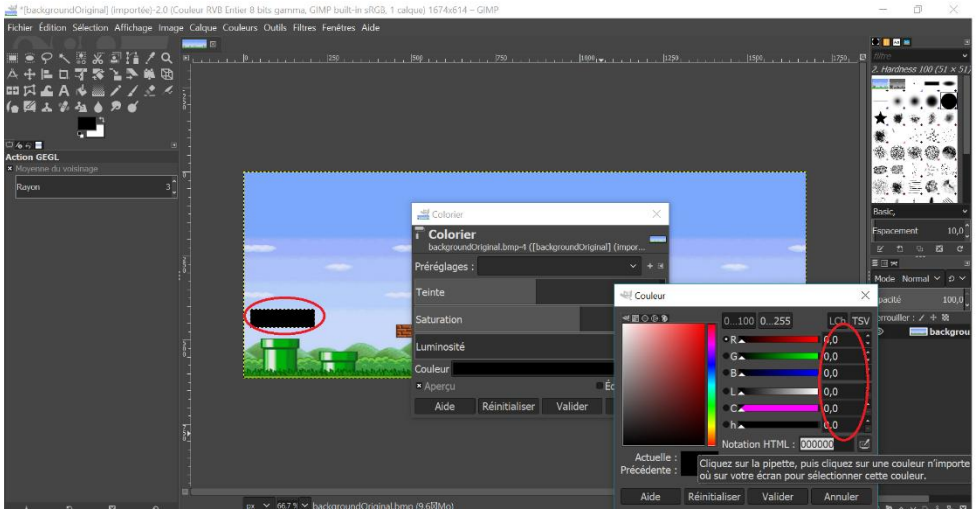


Figure 4: Changing the color of obstacles to black (R=0, G=0, B=0)

- Finally, after the selection and modification of the colors of the obstacles, save the modified image by naming it «backgroundMasque»,then export it in bmp **format** to have at the end an image with the name **«backgroundMasque.bmp».**

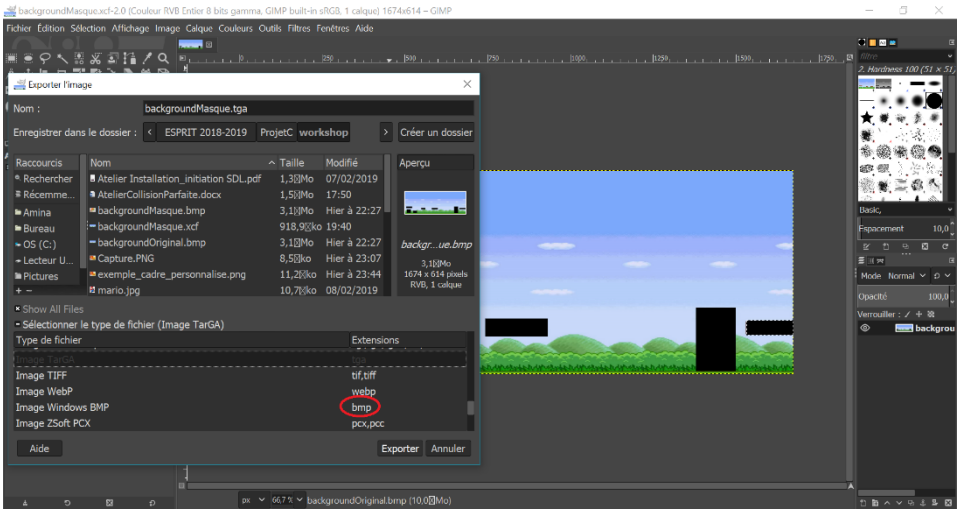


Figure 5: Saving background mask with extension bmp

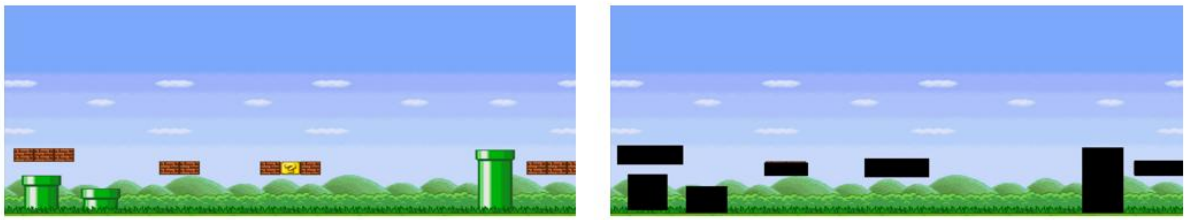
4. Example:

We will use the output of the previous step, which is the mask image, to perform the steps mentioned in paragraph 2 to test the collision.

4.1.Loading backgrounds

Using the SDL 2 library, load both backgrounds (background.bmp and backgroundMasque.bmp)

```
SDL_Surface *bgSurf = SDL_LoadBMP("background.bmp");
SDL_Surface *maskSurf = SDL_LoadBMP("backgroundMasque.bmp");
```



background.bmp backgroundMasque.bmp

Figure 6: Background and backgroundMasque

4.2. Recovery of RGB nuances at a pixel level

Let's say that we have the character Mario, moving it in the background of the game using the keys **SDLK_RIGHT, SDLK_LEFT** et **SDLK_UP**requires the update of the position of the image «mario.bmp» (loaded in memory).

The collision with background obstacles will be perceived at the level of a pixel or more pixels of the image (see the arrows at the level of Mario's photo).

Assuming that:

The character's initial position is:

X= posPerso.x

Y= posPerso.y

The width and height are:

W = posPerso.w = perso->w

H = posPerso. h = perso->h

We propose to choose 8 points (see figure 7) to characterize the outline of Mario's image.

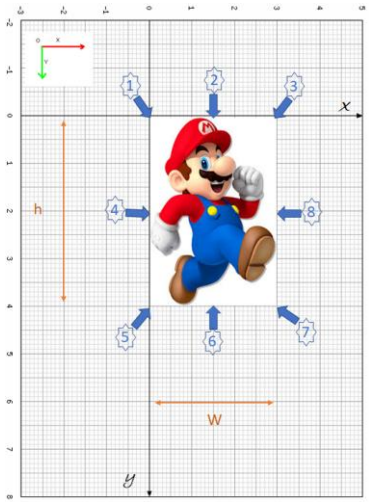


Figure 7: Mario

Subsequently, we can determine the coordinates of the 8 points at which we will detect the collision

Position	x	Y

1	**X**	**Y**
2	**X+W/2**	**Y**
3	**X+W**	**Y**
4	**X**	**Y+H/2**
5	**X**	**Y+H**
6	**X+W/2**	**Y+H**
7	**X+W**	**Y+H**
8	**X+W**	**Y+H/2**

Table 1: Reference pixel coordinates

4.3. Collision detection through RGB shades

- **GetPixel function** [1]

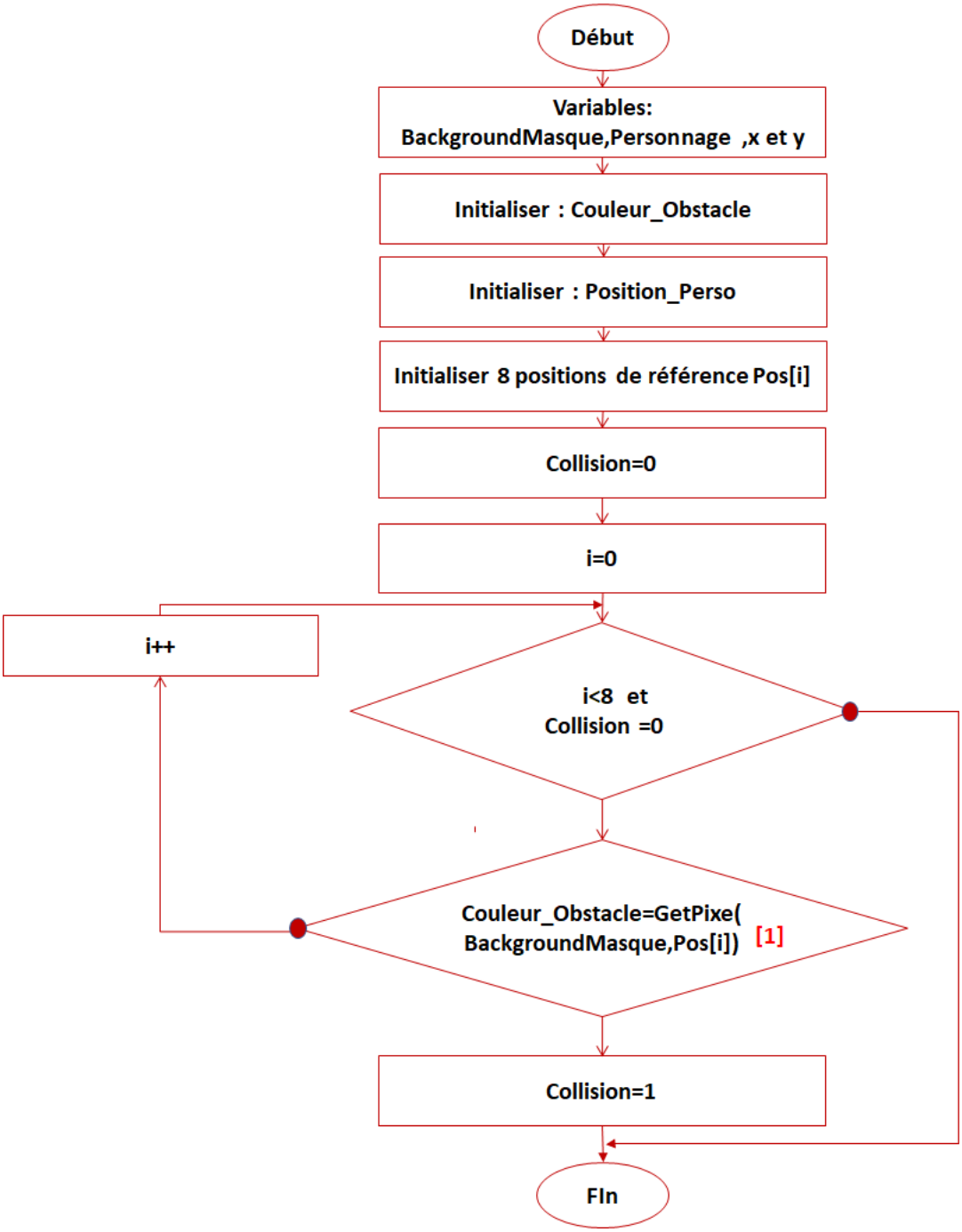
Consider the GetPixel **function** which will return **the color of** a pixel whose **coordinates (x,y),** on a well-defined surface in our case the **backgroundMask .**

- **CollisionPerfect**

Consider the CollisionPerfect **function** will return '1' **in case of collision between** the character **'Mario' in the 8 positions** (See Table 1) **and the backgroundMasque** et **'0'** surface**.**

Below is the organizational chart associated with this function:

```
SDL_Color GetPixel(SDL_Surface *surface, int x, int y) {
    SDL_Color color = {255, 255, 255, 255}; // blanc par défaut
    if (x < 0 || x >= surface->w || y < 0 || y >= surface->h)
        return color;
    Uint8 *pPosition = (Uint8 *)surface->pixels + (y * surface->pitch) + (x * surface->format->BytesPerPixel);
    Uint32 pixel_value = 0;
    memcpy(&pixel_value, pPosition, surface->format->BytesPerPixel);
    SDL_GetRGB(pixel_value, surface->format, &color.r, &color.g, &color.b);
    return color;
}
```



Flow Chart: CollisionParfait Function

5. Work to be done:

Continue with the same work but for the different directions of the left, right, top and bottom character).

°

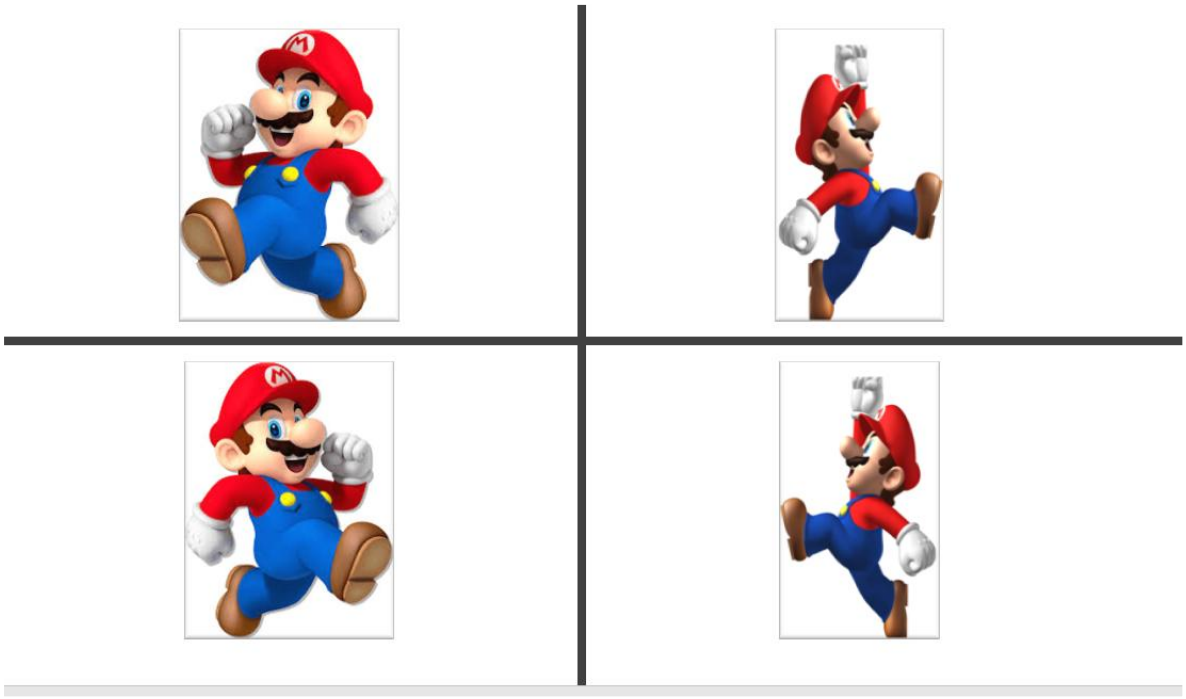


Figure 8: Mario with different directions

NB:

- ✓ Once both backgrounds (Original and Mask) are loaded into memory, you should only display the full forebackground (Original) and above that you display the Mario entity that moves in a well-defined direction.
- ✓ In the scrolling function you have to move both backgrounds (Original and Mask).
- ✓ If there are several types of obstacles you can color them differently so the collision function returns 0 or the number of the obstacle type.